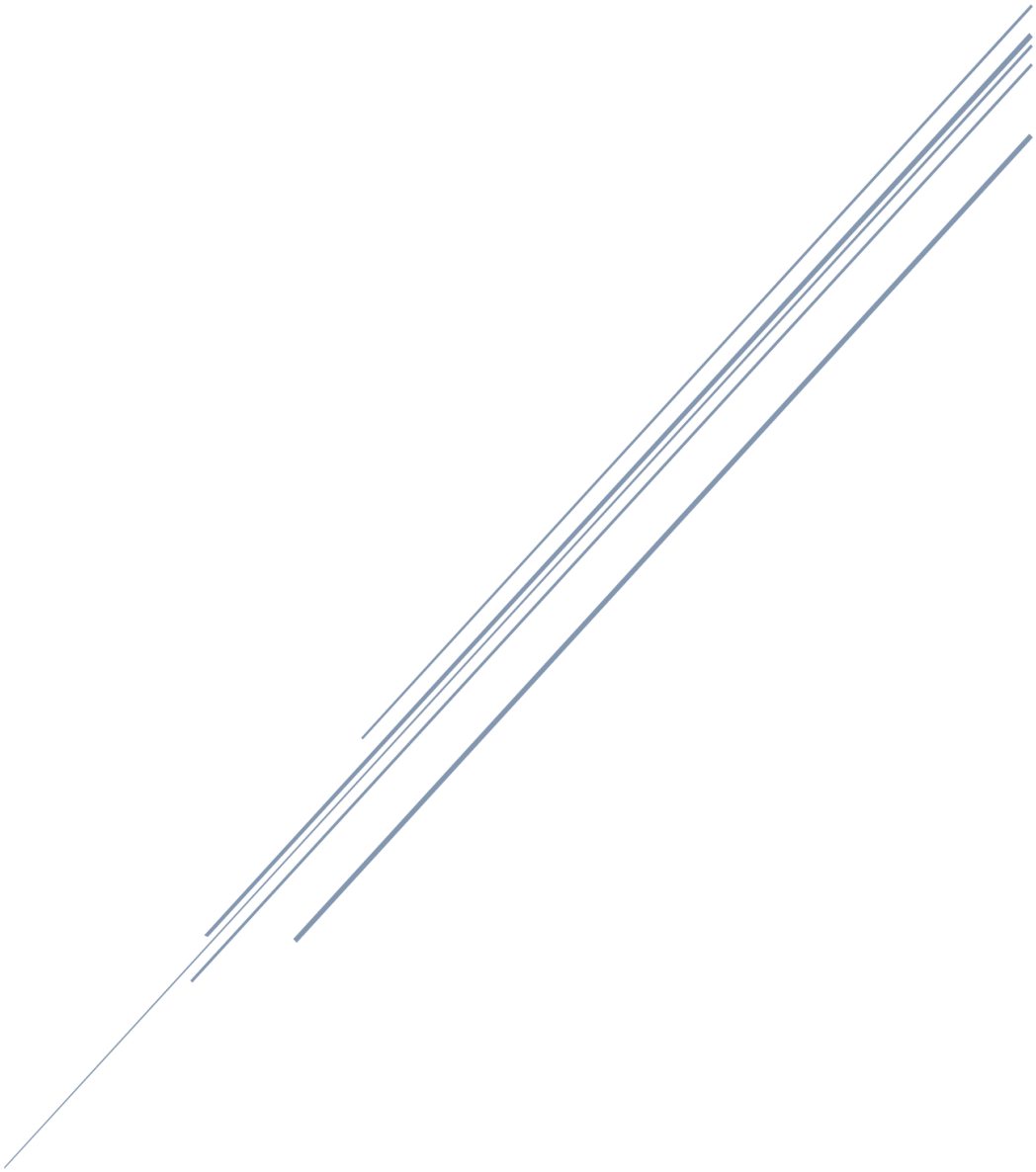
INTERNSHIP DAILY REPORT

Day 4 – Windows and Networking (Attacker View)

CyberShelter Cybersecurity Internship | Week 1



Mohamed Shameem PA

Associate – Information Security (Intern)

12 November 2026

1. Objective

The objective of Day 4 was to understand the basics of cybersecurity, operating systems and networking from an attacker's point of view, and to practise capturing and analysing network traffic. The practical covered packet capture with tcpdump and Wireshark, comparing HTTP and HTTPS traffic, observing DNS queries and the TCP handshake, and service enumeration with nmap on authorised targets.

2. Lab Environment

- **Host Machine:** Windows PC
- **Virtualisation Software:** Oracle VirtualBox (NAT network)
- **Guest OS:** Kali Linux (64-bit), VM "Kali Lab"
- **Tools Used:** tcpdump, Wireshark 4.6, Nmap 7.99, Firefox
- **Authorised Targets:** localhost (127.0.0.1) and scanme.nmap.org

3. Topics Covered

3.1 What is Cybersecurity

Cybersecurity is the practice of protecting computers, networks, applications and data from attacks, damage or unauthorised access. Cyber criminals attack for money, information, disruption or political reasons. A cybersecurity professional's job is to find weaknesses before attackers do, and to detect and respond to attacks when they happen.

- **CIA Triad: Confidentiality** means only authorised people can see the data, **Integrity** means the data is not changed without permission, and **Availability** means systems are available when needed.
- **Types of Attackers:** cyber criminals (money), hacktivists (political or social reasons), nation-state actors (spying) and insiders (employees misusing access).
- **Offense vs Defense:** the **Red Team** (penetration testers) legally attacks systems to find weaknesses, while the **Blue Team** (SOC analysts) monitors, detects and responds to attacks.

3.2 Operating Systems (Security View)

An operating system manages the computer's hardware and software, and it is often the main target for attackers. Both Windows and Linux use users, permissions, processes and logs to stay secure, but they manage them in different ways.

Feature	Windows	Linux
Admin account	Administrator	root
Permissions	NTFS permissions, UAC	rwx permissions, sudo
Logs	Event Viewer	/var/log, journalctl
Main use	Desktops and company networks (Active Directory)	Servers, cloud and security tools

- **Why Attackers Target the OS:** missing updates (known vulnerabilities), weak passwords, too many admin accounts, and unnecessary services and open ports.

3.3 Networking Fundamentals

Networking is how devices connect and exchange data. To attack or defend a network, it is important to understand how data moves between devices. This is explained using the **OSI model**, which splits communication into 7 layers.

Layer	Name	Example
7	Application	HTTP, DNS
6	Presentation	Encryption (TLS)
5	Session	Session management
4	Transport	TCP, UDP (ports)
3	Network	IP addresses, routers
2	Data Link	MAC addresses, switches
1	Physical	Cables, Wi-Fi signals

- **Network Devices:** a **switch** connects devices in a LAN using MAC addresses, a **router** connects different networks using IP addresses, and **ARP** finds the MAC address for an IP address.
- **TCP, UDP & Ports:** TCP is reliable and starts with a **3-way handshake (SYN → SYN-ACK → ACK)**, while UDP is faster with no handshake. Common ports are 21 FTP, 22 SSH, 53 DNS, 80 HTTP, 443 HTTPS and 3389 RDP. Attackers use **port scanning** to find open ports and running services.

3.4 IP, DNS, HTTP/HTTPS

These are the core protocols used every time a website is opened. An **IP address** identifies the server, **DNS** finds that IP from the website name, and **HTTP or HTTPS** carries the web page between the browser and the server.

- **IP Address:** IPv4 looks like 192.168.1.10. Private IPs (10.x.x.x, 172.16–31.x.x, 192.168.x.x) are used inside local networks, and **NAT** lets many private devices share one public IP.
- **DNS:** converts names like google.com into IP addresses using **UDP port 53**. Attackers can misuse DNS for **DNS spoofing**, which sends victims to fake websites.
- **HTTP vs HTTPS:** HTTP (port 80) sends data in **plain text**, so anyone on the network can read it. HTTPS (port 443) encrypts the data with **TLS**, so captured traffic cannot be read.

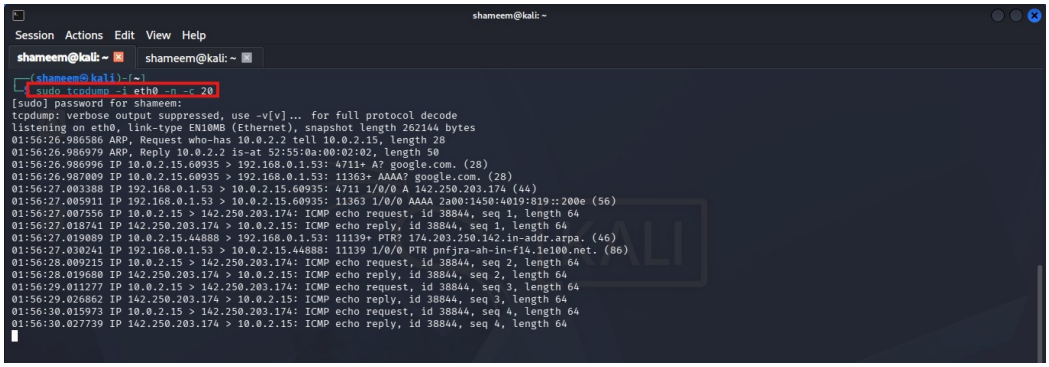
4. Tasks Performed

All tasks were performed in the Kali Linux VM. Only authorised targets were scanned: localhost and scanme.nmap.org, which is provided by the Nmap project for testing.

4.1 Packet Capture with tcpdump

4.1.1 Capturing Packets

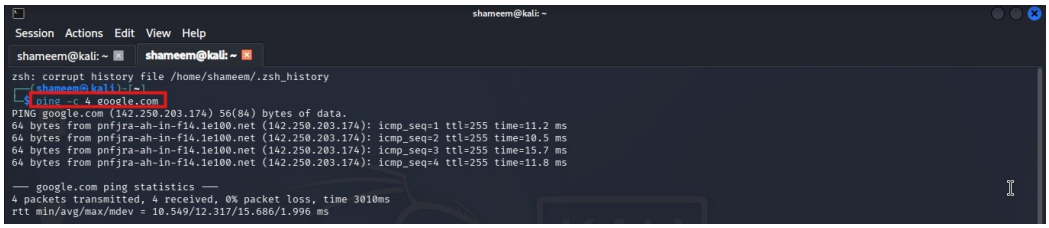
- **Command Executed:** `sudo tcpdump -i eth0 -n -c 20`
- **Purpose:** Captures 20 packets on the eth0 interface and shows IP addresses instead of names.
- **Observation:** The capture shows an **ARP** request for the gateway 10.0.2.2, **DNS queries** from 10.0.2.15 to the DNS server 192.168.0.1 for google.com (A and AAAA records), the DNS reply **142.250.203.174**, and **ICMP echo request and reply** packets from the ping test.



Packets captured with tcpdump.

4.1.2 Generating Traffic

- **Command Executed:** `ping -c 4 google.com`
- **Purpose:** Sends 4 ping packets from a second terminal to create traffic for the capture.
- **Observation:** google.com replied to all 4 packets with **0% packet loss** and an average time of about 12 ms.



Ping to google.com.

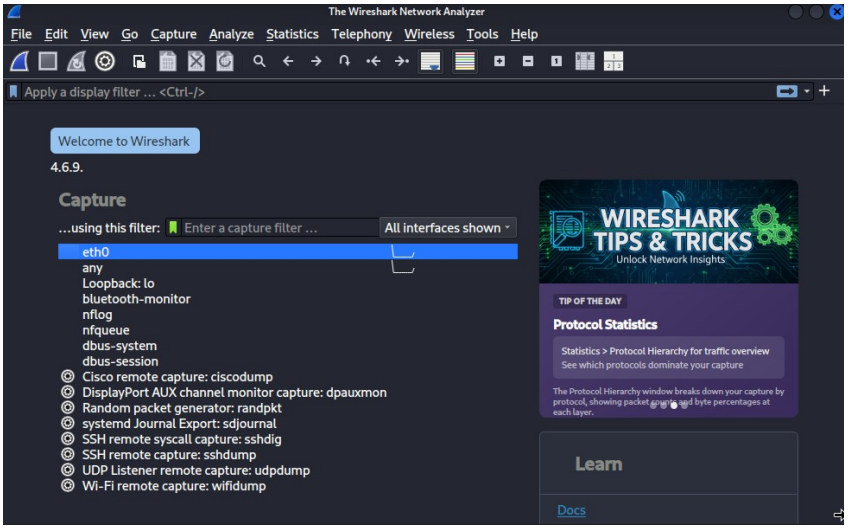
4.2 Wireshark

4.2.1 Starting Wireshark

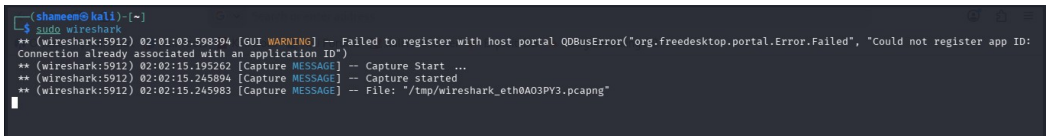
- **Command Executed:** `sudo wireshark`
- **Purpose:** Opens Wireshark with permission to capture network traffic.
- **Observation:** Wireshark 4.6.9 opened and listed the available interfaces. **eth0** was selected and the capture started, saving packets to a temporary capture file.



Starting Wireshark.



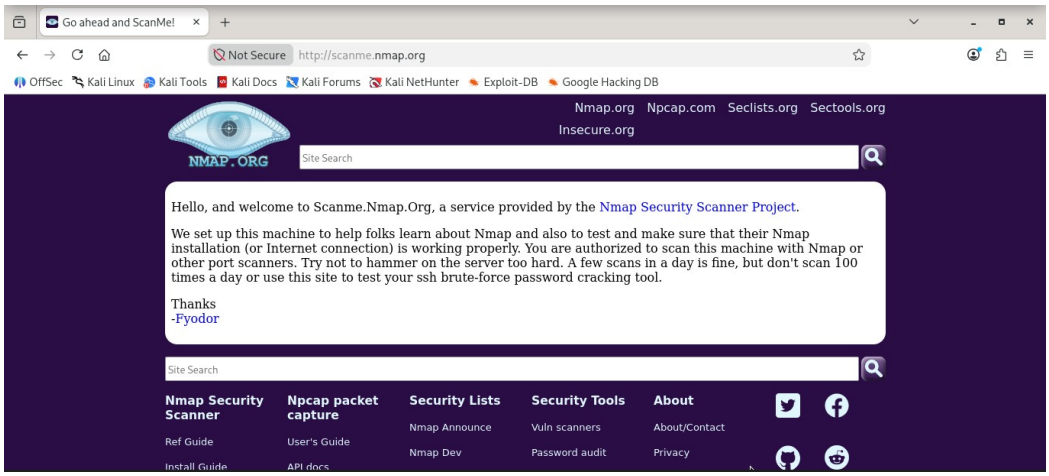
Selecting the eth0 interface.



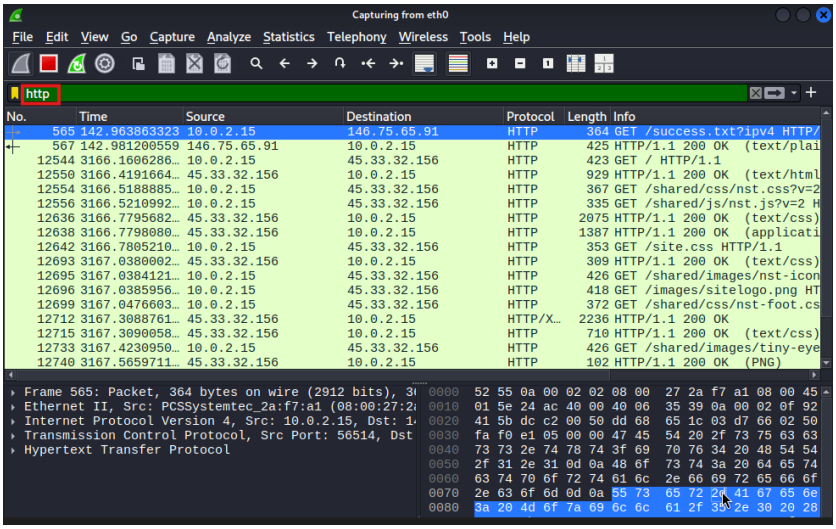
Capture started on eth0.

4.2.2 HTTP Traffic (Not Encrypted)

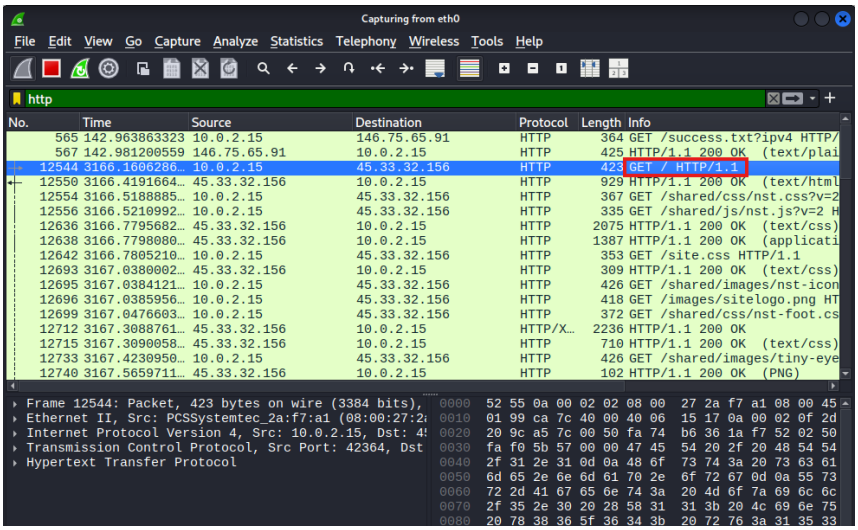
- **Action:** Opened `http://scanme.nmap.org` in Firefox and applied the Wireshark filter `http`.
- **Purpose:** Captures web traffic that is sent without encryption.
- **Observation:** Firefox showed **"Not Secure"** for the site. Wireshark captured the **GET / HTTP/1.1** request from 10.0.2.15 to 45.33.32.156 and the **HTTP/1.1 200 OK** response. Expanding the request showed every header in **plain text**, including the Host (scanme.nmap.org), User-Agent, Accept-Language and Referer.



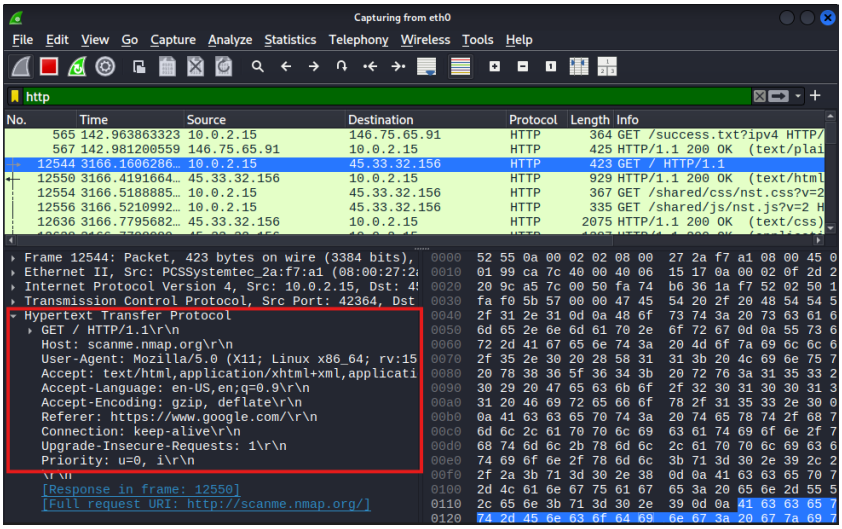
scanme.nmap.org loaded over HTTP.



HTTP packets filtered in Wireshark.



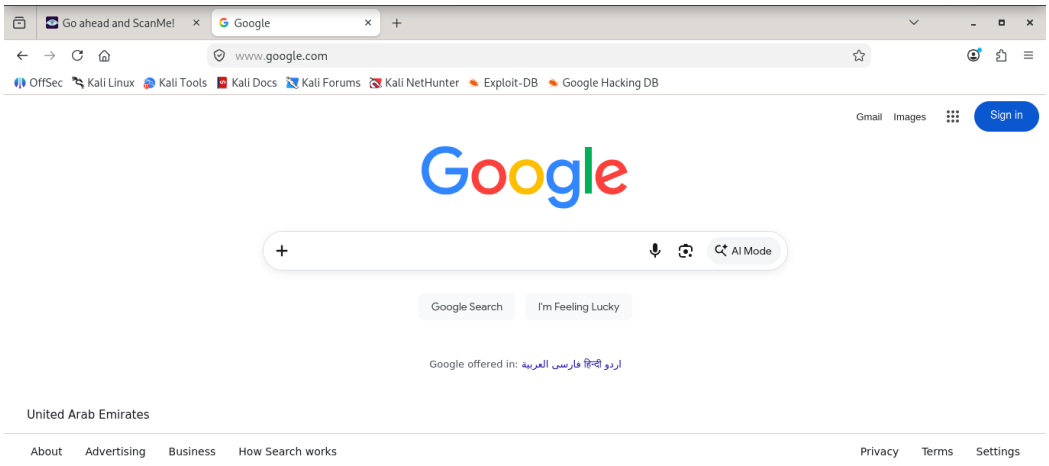
The HTTP GET request.



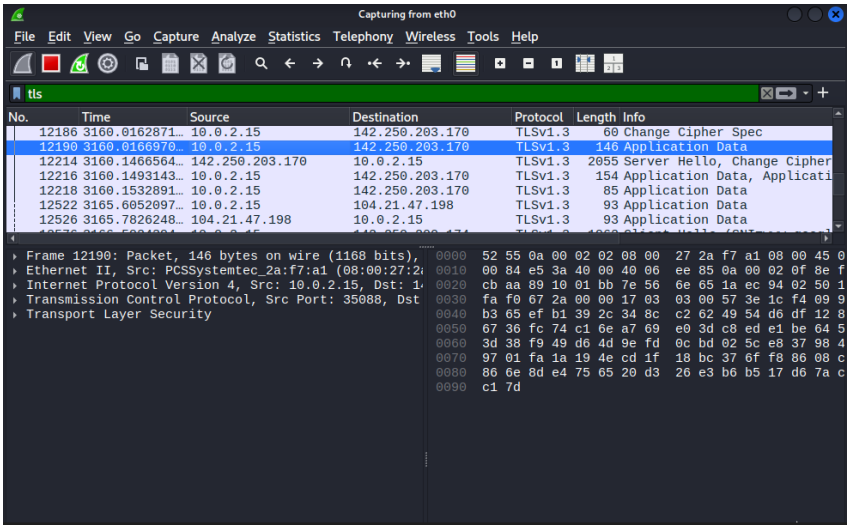
HTTP request headers readable in plain text.

4.2.3 HTTPS Traffic (Encrypted)

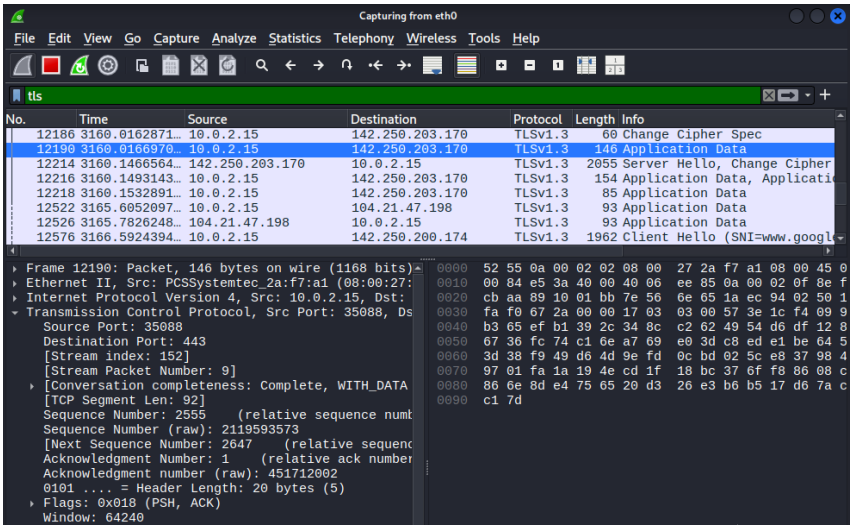
- **Action:** Opened `https://www.google.com` and applied the filter `tls`.
- **Purpose:** Captures web traffic that is protected by TLS encryption.
- **Observation:** The packets are shown as **TLSv1.3** with **Application Data**, sent to **Destination Port 443**. Unlike HTTP, the content of the request cannot be read, because it is encrypted. Only the connection details, such as IP addresses and ports, are visible.



google.com loaded over HTTPS.



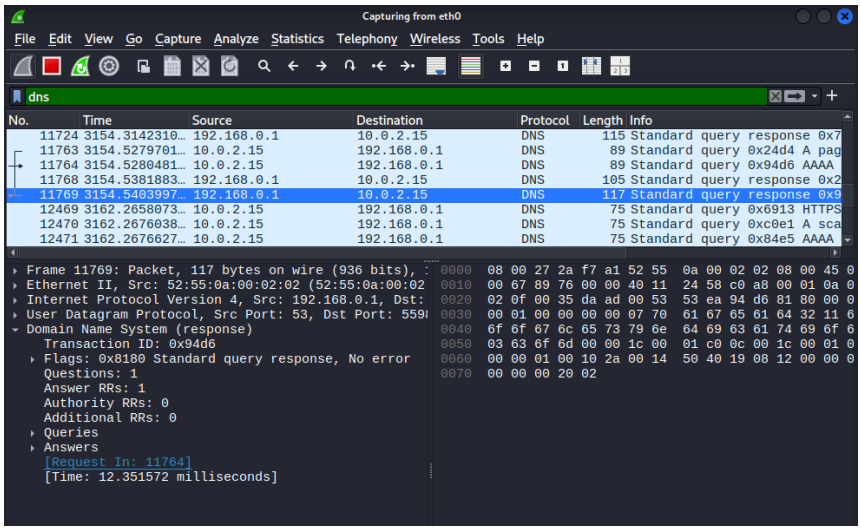
TLS packets filtered in Wireshark.



Encrypted traffic on port 443.

4.2.4 DNS Queries

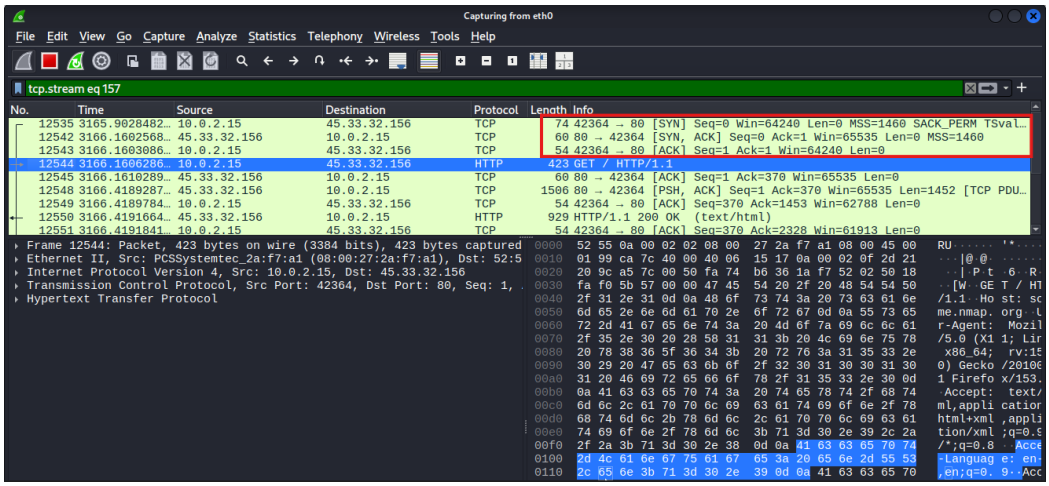
- **Filter Used:** `dns`
- **Purpose:** Shows the DNS requests the system makes to find IP addresses.
- **Observation:** Kali (10.0.2.15) sent **Standard query** packets to the DNS server **192.168.0.1**, and the server replied with a **Standard query response, No error**. The response used **UDP source port 53**, had 1 question and 1 answer, and arrived in about 12 ms.



DNS query and response.

4.2.5 TCP 3-Way Handshake

- **Filter Used:** `tcp.stream eq 157` (Conversation Filter on the HTTP request)
- **Purpose:** Shows how a TCP connection is set up before data is sent.
- **Observation:** Before the HTTP GET request, three packets set up the connection between Kali (port 42364) and scanme.nmap.org (port 80): **[SYN]** from Kali, **[SYN, ACK]** from the server, and **[ACK]** from Kali. The GET request was sent only after this handshake was complete.

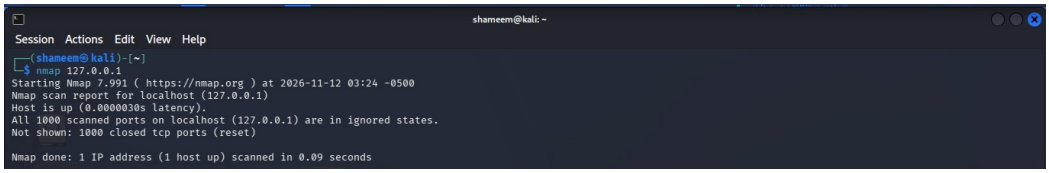


TCP 3-way handshake before the HTTP request.

4.3 Service Enumeration with Nmap

4.3.1 Scanning Localhost

- **Command Executed:** `nmap 127.0.0.1`
- **Purpose:** Scans the 1000 most common ports on my own machine.
- **Observation:** The host is up, and **all 1000 scanned ports are closed**. This matches Day 3, where no services were listening on the system.



Nmap scan of localhost.

4.3.2 Service Scan of scanme.nmap.org

- **Command Executed:** `sudo nmap -sS -sV -F scanme.nmap.org`
- **Purpose:** Runs a SYN (stealth) scan with service version detection on the top 100 ports of the authorised test server.
- **Observation:** The scan used the IP 45.33.32.156. Port **22/tcp is open** (shown as tcpwrapped), while ports **25, 80, 873, 1720 and 5060** are **filtered**, meaning a firewall or the network dropped the scan packets. 94 ports were closed.



Nmap service scan of scanme.nmap.org.

5. Results

5.1 Hands-on Task Findings

Item	Result
DNS Request	Standard query from 10.0.2.15 to 192.168.0.1, response on UDP port 53
TCP Handshake	SYN → SYN-ACK → ACK between 10.0.2.15:42364 and 45.33.32.156:80
HTTP Traffic	GET request and headers readable in plain text
HTTPS Traffic	TLSv1.3 Application Data on port 443, content encrypted
Localhost Scan	All 1000 ports closed
scanme Scan	Port 22 open, ports 25, 80, 873, 1720, 5060 filtered

5.2 Why HTTPS is Secure

HTTPS uses **TLS encryption** to protect data between the browser and the server. It provides **confidentiality** (the data is encrypted), **integrity** (the data cannot be changed without being detected) and **authentication** (a certificate proves the website is genuine). This was clear in the practical: the HTTP request to scanme.nmap.org could be read line by line in Wireshark, while the HTTPS traffic to google.com only showed encrypted Application Data.

6. Key Learnings

- Learned how to capture packets with `tcpdump` and analyse them with Wireshark filters like `http`, `tls` and `dns`.
- Saw the difference between HTTP and HTTPS in real traffic, and why HTTP should never be used for sensitive data.
- Identified a DNS request and a TCP 3-way handshake in captured traffic.
- Learned how nmap SYN scans and service detection work, and why only authorised targets should be scanned.

7. Challenges Faced

- **testphp.vulnweb.com did not load:** the site was most likely blocked by the local internet provider, so HTTP traffic was captured using http://scanme.nmap.org instead.
- **Slow nmap scan:** the full scan of scanme.nmap.org showed "retransmission cap hit" and was very slow, because scan packets were dropped through the VirtualBox NAT network. A faster scan with `-F` (top 100 ports) was used instead.

8. Conclusion & Next Steps

Day 4 showed how attackers and defenders look at network traffic. Capturing packets made it clear why HTTPS is important, and the nmap scans showed how open and filtered ports are discovered. **Next Steps:** Day 5 will focus on cybersecurity basics from both the offense and defense side.